



MINICURSO DE RUBY

PROGRAMAÇÃO COM RUBY

Da sintaxe básica até consumir uma API real, e entender por que Ruby permite frameworks como Rails.



www.carlospohlod.com

Carlos Daniel Pohlod





SOBRE MIM

- Ciência da Computação pela **Unicentro**
- Há ~5 anos programando **Ruby** profissionalmente
- Sempre via consultoria: um time, vários clientes e domínios diferentes
- Conteúdo sobre Ruby e Rails no YouTube e no LinkedIn
- Tenho commits no Rails e no Ruby



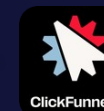
HOJE



LendingOne: crédito imobiliário para investidores nos EUA



2021 A 2025



ClickFunnels, AgriSafe e a.karta: marketing, crédito rural e pesquisa





O PLANO

ATO	TEMA	TEMPO
1	Que linguagem é essa?	~15 min
2	Começando a pensar em Ruby	~50 min
3	Ruby no mundo real: gems, HTTP, uma API de verdade	~45 min
4	Metaprogramação (avançado)	~15 min

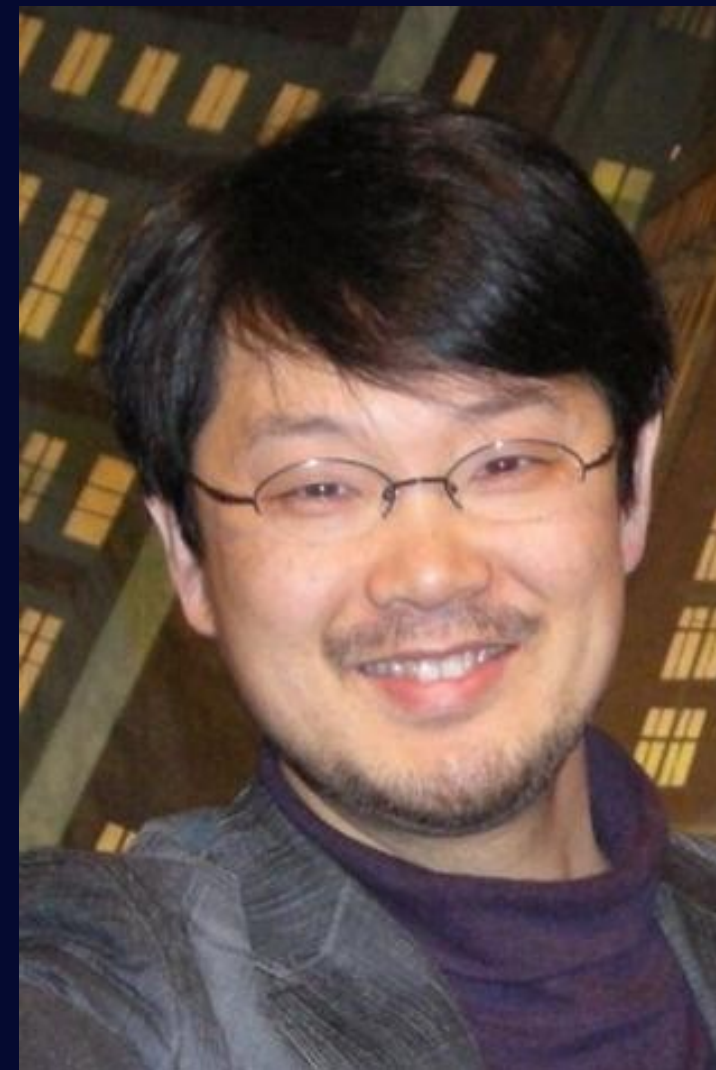




YUKIHIRO "MATZ" MATSUMOTO

"Ruby é projetado para deixar programadores felizes." Matz, criador do Ruby (1995, Japão)

- Sintaxe pensada para **peessoas**, não para a máquina
- Produtividade e *developer happiness* como princípio
- Open source, comunidade forte até hoje





FEITO PARA O PROGRAMADOR, NÃO PARA A MÁQUINA

```
main.rb  
  
puts("curso de ruby")  
puts "curso de ruby"  
p "curso de ruby"
```

Lê-se quase como texto: sem **;**, sem tipos, sem parênteses obrigatórios.



E tem o Rails.

- Framework web completo, nascido do Ruby em 2004
- Convenção no lugar de configuração
- GitHub, Shopify e Airbnb nasceram nele





QUEM USA RUBY?

GitHub

a maior plataforma
de código do mundo



e-commerce, grande
apoiador do Ruby



Ruby em ferramentas e
serviços internos

stripe

pagamentos



DevOps



airbnb hospedagem

37signals



Basecamp e HEY,
criadores do Rails



atendimento



ClickFunnels
ClickFunnels
marketing

Quase sempre via **Ruby on Rails**, o framework que fez Ruby ficar famoso. Voltamos nele no final.





EXECUTANDO RUBY

```
$ ruby --version  
ruby 3.2.5 (2024-07-26)
```

```
$ cat hello.rb  
puts "Olá, Ruby!"
```

```
$ ruby hello.rb  
Olá, Ruby!
```

```
$ irb  
irb(main):001> "ruby".upcase  
=> "RUBY"  
irb(main):002> 5.times { puts "Ruby" }  
Ruby  
Ruby  
...
```

Arquivo **.rb** para programas; **IRB** (Interactive Ruby) para experimentar. Vamos usar os dois hoje.





TIPOS BÁSICOS DE DATOS



main.rb

```
numero = 1  
numero_real = 1.0  
string = "Olá, mundo!"  
array = [1, 2, 3]
```





SPOILER: EM RUBY, **TUDO É UM OBJETO**

```
main.rb

class Integer
end

# 1 é uma instancia de Integer

p 1.class

# textos (string) são objetos da class String

class String
end

p "um texto qualquer".class
```





ARRAYS E HASHES

```
linguagens = ["Ruby", "Python", "Go"]
```

```
linguagens.first # => "Ruby"
```

```
linguagens.size # => 3
```

```
developer = {  
  name: "Carlos",  
  language: "Ruby"  
}
```

```
developer[:name]  
# => "Carlos"
```

Hashes são muito parecidos visualmente com JSON. Repare também que Ruby tem uma forma especial de acessar chaves de um hash, como `":name"`





EXERCÍCIO RÁPIDO

Crie um array com dois valores: a string “ruby” e o hash “developer”.
Depois, com puts, imprima os dois. Saída esperada:

```
linguagens = ["Ruby", "Python", "Go"]
```

```
# => "Ruby"
```

```
# => {
```

```
  name: "Carlos",
```

```
  language: "Ruby"
```

```
}
```





CONTROLE DE FLUXO

FUNDAMENTAL

```
if idade >= 18
  puts "Maior de idade"
else
  puts "Menor de idade"
end
```

RUBY IDIOMÁTICO

```
puts "Pode entrar" if idade >= 18

puts "Bloqueado" unless ativo

return unless user.active?
```

Só `nil` e `false` são "falsos". Todo o resto é "verdadeiro", inclusive `0` e `""`.





MÉTODOS

```
def imprimir(nome)
  p nome
end

imprimir("Ruby")
# => "Ruby"
```

`def` define, parâmetros entre parênteses, e a **última expressão é o retorno**, sem `return` explícito.





KEYWORD ARGUMENTS

```
def criar_developer(name:, language:)  
  "#{name} programa em #{language}"  
end  
  
criar_developer(name: "Carlos", language: "Ruby")  
# => "Carlos programa em Ruby"
```

Parece o Hash de antes? **Não é coincidência.** Argumentos nomeados deixam a chamada legível. Rails usa isso em todo lugar.





ATO 2

COMEÇANDO A PENSAR EM RUBY

Iterar, passar comportamento para métodos, transformar coleções. Só então
organizar tudo em classes.





FOR FUNCIONA...

```
for numero in [1, 2, 3]
  puts numero
end
```

Funciona. Mas quase ninguém escreve assim.

```
[1, 2, 3].each do |numero|
  puts numero
end
```

Ruby normalmente é escrito assim: **o Array itera a si mesmo.**





BLOCOS: INICIALMENTE PODE SER CONFUSO

main.rb

```
def executar  
  yield  
end  
  
executar do  
  puts "Olá, Ruby!"  
end
```

`yield` executa o block recebido. Você pode imaginar que o “yield” é substituído pelo código que você enviou para ele





VOCÊ PODE DISPONIBILIZAR VARIÁVEIS DE DENTRO PARA FORA EM BLOCOS

```
def executar
  yield
end

executar do
  puts "Olá, Ruby!"
end
```

```
def executar
  # a string "ruby" vai ser enviada para a
  variável "linguagem"
  yield("Ruby")
end

executar do |linguagem|
  "Olá, #{linguagem}!"
end

# resultado: Olá, Ruby!
```

`yield` executa o block recebido. Pode **passar valores** para ele e **receber** o que ele devolve. É exatamente o que `each` faz por baixo.





EXERCÍCIO

Faça um método com bloco chamado calcular. O método exibe o texto “O resultado é:” e, logo após, tem um yield que receberá uma expressão de soma de dois valores.





SOLUÇÃO



main.rb

```
def calcular  
  puts "O resultado é:"  
  yield  
end  
  
calcular { puts 10 + 20 }
```





É POR ISSO QUE RUBY TEM ESSA CARA

```
3.times do
  puts "Ruby"
end
```

```
# Rails
User.transaction do
  user.save!
  account.debit!(100)
end
```

Não é sintaxe especial. É um método (`transaction`) recebendo um block.

Ruby permite **passar comportamento** para métodos.





ENUMERABLE: OS TRÊS QUE VOCÊ MAIS VAI USAR

```
numbers = [1, 2, 3, 4, 5]

numbers.map { |number| number * 2 }      # => [2, 4, 6, 8, 10]
numbers.select { |number| number.even? }  # => [2, 4]
numbers.find { |number| number > 3 }      # => 4
```

- **map** transforma cada item e devolve um novo Array
- **select** filtra pelos itens em que o block é verdadeiro
- **find** devolve o primeiro item em que o block é verdadeiro





DESAFIO 1 · 2 MINUTOS

COMO RETORNAR SÓ OS NÚMEROS PARES, MULTIPLICADOS POR 10?



main.rb

```
numbers = [1, 2, 3, 4, 5, 6]
```



+

=

TUDO NO RUBY É ORIENTADO A OBJETOS, ATÉ TIPOS DE DADOS!

```
class String
  def dizer_algo
    puts "adicionei um metodo dentro da string!!"
  end
end

"um texto qualquer".dizer_algo
```

```
class Integer
  def fazer_algo
    puts "fiz algo"
  end
end

1.fazer_algo
```





CLASSES

```
main.rb

class Curso
  attr_accessor :nome

  def initialize(nome)
    @nome = nome
  end
end

curso = Curso.new("Ruby")
curso.nome
```



REGRAS PARA ACESSAR ATRIBUTOS

```
class Curso
  attr_accessor :nome
  attr_reader :duracao

  def initialize(nome, duracao, valor)
    @nome = nome
    @duracao = duracao
    @valor = valor
  end

  private

  attr_reader :valor
end

curso = Curso.new("Ruby", "3 meses", 1000)
puts curso.duracao
puts curso.valor # Vai gerar um erro, pois o atributo é privado
curso.duracao = "4 meses" # Vai gerar um erro, pois o atributo é somente leitura
```





HERANÇA

```
main.rb

class Curso
  attr_accessor :nome
  attr_reader :duracao

  def initialize(nome, duracao)
    @nome = nome
    @duracao = duracao
  end
end

class Ruby < Curso
  def info
    puts "Curso de #{nome} com duração de #{duracao}"
  end
end

curso = Ruby.new("Ruby", "3 meses")
curso.info
```



MÉTODOS DA CLASSE E DE INSTÂNCIA



```
main.rb

class Curso
  def self.metodo_de_classe
    puts "Método de classe"
  end

  def metodo_de_instancia
    puts "Método de instância"
  end
end

Curso.metodo_de_classe
puts Curso.public_methods.include?(:metodo_de_classe)

curso = Curso.new
curso.metodo_de_instancia
puts curso.methods.include?(:metodo_de_instancia)
```





ATO 3

RUBY NO MUNDO REAL

Até aqui: objetos, métodos, blocks, Enumerable, classes e módulos. Agora: gems, HTTP, JSON e uma API de verdade.





O QUE É UMA GEM?



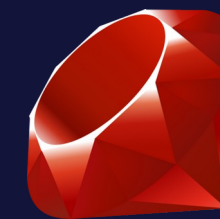
npm / yarn
Node.js



pip
Python



Maven / Gradle
Java



gems
Ruby

Toda linguagem tem um jeito de instalar bibliotecas prontas. Em Ruby, cada biblioteca é uma **gem**: um pacote de código que você instala com um comando e usa no seu programa.

RubyGems (rubygems.org) é o repositório com 180 mil+ gems. **Bundler** é a ferramenta que instala e trava as versões das gems do seu projeto.





GEMS E BUNDLER

```
main.rb

# Gemfile
source "https://rubygems.org"

gem "faraday"
```

```
bash

$ bundle install
```

```
main.rb

# app.rb
require "faraday"
require "json"
```

Gem = biblioteca empacotada pela comunidade. **Bundler** instala e trava as versões (**Gemfile.lock**).

Rails é uma gem. Faraday é uma gem.
json já vem com o Ruby.





HTTP, API E JSON

seu script



GET /users



API



resposta JSON

```
[
  {
    "id": 1,
    "name": "Leanne Graham",
    "email": "Sincere@april.biz",
    "address": { "city": "Gwenborough" }
  }
]
```

Uma API responde texto em **JSON**.
Repare na forma: parece um **Array de Hashes**.

Vamos usar
jsonplaceholder.typicode.com, uma
API pública de testes.





CHEFÃO FINAL: CONSUMINDO UMA API REAL

```
require "faraday"
require "json"

response = Faraday.get("https://jsonplaceholder.typicode.com/users")
response.status      # => 200
response.body        # => "[{"id": 1, "name": ...}" (uma String)

users = JSON.parse(response.body)
users.class          # => Array
users.size           # => 10
users.first.class    # => Hash
```





A RESPOSTA É UM ARRAY DE HASHES...

```
main.rb  
  
users.each do |user|  
  puts user["name"]  
end
```

```
Leanne Graham  
Ervin Howell  
Clementine Bauch  
...
```

Chaves em **String** ("name"), porque vieram do JSON. Mesmo **each**, mesmo block, mesmo **|user|** de antes.





DESAFIO 2 · 3 MINUTOS

QUANTOS USUÁRIOS TÊM E-MAIL TERMINANDO EM **.BIZ**?

```
users = JSON.parse(response.body)
```

Dica: strings respondem a **end_with?(".biz")**. E existe um método que conta.





RESPOSTA

FUNDAMENTAL

```
main.rb

biz = users.select do |user|
  user["email"].end_with?(".biz")
end

biz.size    # => 3
```

GOING DEEPER

```
main.rb

users.count do |user|
  user["email"].end_with?(".biz")
end

# => 3
```

Enumerable tem dezenas de métodos assim: `count`, `sum`, `sort_by`, `group_by`, `min_by`... Todos recebem um block.





E QUANDO DÁ ERRADO? EXCEPTIONS

```
begin
  10 / 0
rescue ZeroDivisionError
  puts "Divisão por zero!"
end
```

```
begin
  response = Faraday.get(url)
rescue Faraday::Error => error
  puts "Erro: #{error.message}"
end
```

`begin ... rescue ... end` captura o erro e deixa o programa continuar. Numa request, a rede pode falhar: proteja o que você não controla.





ATO 4 · BÔNUS

METAPROGRAMAÇÃO, ONDE ESTÁ A MALDADE DO RUBY

Por que Ruby é poderoso





OPEN CLASSES

```
class String
  def shout
    upcase + "!"
  end
end

"ruby".shout    # => "RUBY!"
```

Classes nunca "fecham": dá para reabrir até as da própria linguagem.
Poderoso, e perigoso.





METAPROGRAMAÇÃO: CÓDIGO QUE ESCREVE CÓDIGO

```
class User
end

User.class_eval do
  def hello
    "Hello!"
  end
end

User.new.hello # => "Hello!"
```

```
class User
  [:name, :email, :age].each do |attribute|
    define_method(attribute) do
      instance_variable_get("@#{attribute}")
    end
  end
end
```

Três métodos criados por um **each**. Lembra do **attr_reader**? É assim que ele funciona.





RAILS É O PRÓXIMO PASSO (MAS NÃO O ÚNICO)

```
class User < ApplicationRecord
  has_many :posts
end

User.where(active: true)
User.find_by(email: "ana@example.com")
user.posts.map(&:title)
```

Rails: o uso mais famoso de Ruby. Web, banco, jobs, e-mail, tudo com essa cara.

Ruby também é ótimo para:

- scripts e **automação**
- **CLIs** (Rails, Bundler e o Homebrew são CLIs em Ruby)
- **APIs** e serviços
- scraping e processamento de dados
- criar suas próprias **gems**





PARA ONDE IR AGORA?

docs.ruby-lang.org

documentação oficial (também em ruby-lang.org/pt)

try.ruby-lang.org

Ruby no navegador, sem instalar nada

exercism.org/tracks/ruby

exercícios com mentoria, gratuito

guides.rubyonrails.org

Rails Guides. Comece por "Getting Started"

Ruby Under a Microscope

Pat Shaughnessy. Internals, para quem quer ir fundo

rubyapi.org

referência rápida e bonita da API



OBRIGADO!

Perguntas?

contact@carlospohlod.com · carlospohlod.com

